

Docker com PySpark na Prática

Setup completo — do zero ao notebook rodando no VS Code dentro do container

Matheus · 2026

Este guia documenta o processo completo de configurar Docker em um projeto PySpark — escolha de imagem, Dockerfile, Docker Compose, Dev Containers e integração com VS Code.

Parte 1 — Escolhendo a imagem certa

Como encontrar a imagem ideal

A escolha da imagem base é o passo mais importante. Uma imagem errada gera horas de debug de Java, variáveis de ambiente e compatibilidade. O processo correto é em três etapas.

1. Pesquisa no Docker Hub

Acesse hub.docker.com e pesquise o nome da tecnologia. Para PySpark, as opções principais são `apache/spark` (oficial do projeto Apache) e imagens da comunidade.

2. Verifique antes de adotar

O que verificar	Por que importa
Data do último update	Imagem abandonada = sem correções de segurança e bugs
Número de downloads	Quanto mais usado, mais testado pela comunidade
Tags disponíveis	Garante que a versão que você precisa existe
Se está marcada como 'supported'	Imagens depreciadas podem parar de funcionar

3. Entenda as tags

As tags da imagem `apache/spark` parecem complexas mas são simples — cada parte indica uma versão:

```
apache/spark:4.0.2-scala2.13-java17-python3-r-ubuntu
| | | | |
| | | | S0 base
| | | Python incluído
| | Java 17
| Scala 2.13
Spark 4.0.2
```

A imagem escolhida

`apache/spark:4.0.2-scala2.13-java17-python3-r-ubuntu` Java 17 (mais estável que 21 para PySpark), Python incluído, mantida pelo projeto Apache oficial. Sem necessidade de instalar Java manualmente.

Baixando a imagem

```
docker pull apache/spark:4.0.2-scala2.13-java17-python3-r-ubuntu
```

O `docker pull` pode ser rodado em qualquer diretório — ele baixa a imagem para o Docker da sua máquina, não para uma pasta específica. Imagens ficam armazenadas internamente pelo Docker.

Parte 2 — Estrutura do projeto

Arquivos necessários

Na raiz do projeto, criar os seguintes arquivos:

Arquivo	Função
Dockerfile	Define como a imagem do projeto é construída
docker-compose.yml	Orquestra os containers e suas configurações
requirements.txt	Lista as dependências Python do projeto
.dockerignore	Define o que não deve ser copiado para a imagem
.devcontainer/devcontainer.json	Configura a integração com VS Code

.dockerignore

```
.git
__pycache__
*.pyc
.env
```

Evita copiar para dentro da imagem arquivos desnecessários como cache do Python, arquivos git e variáveis de ambiente sensíveis.

Dockerfile

```
FROM apache/spark:4.0.2-scala2.13-java17-python3-r-ubuntu

USER root

WORKDIR /app

COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

CMD ["bash"]
```

Instrução	O que faz
FROM apache/spark:...	Usa a imagem oficial do Spark como base — Java e Python já incluídos
USER root	Necessário nessa imagem — o usuário spark padrão não tem permissão para instalar pacotes

WORKDIR /app	Define /app como diretório de trabalho. Convenção do mercado — você escolhe o nome
COPY requirements.txt .	Copia só o requirements primeiro — aproveita o cache do Docker
RUN pip3 install ...	Instala as dependências. pip3 porque a imagem usa python3
COPY . .	Copia o restante do projeto respeitando o .dockerignore
CMD ["bash"]	Abre terminal bash quando o container sobe

Por que USER root?

A imagem apache/spark roda por padrão com o usuário 'spark', que não tem permissão para instalar pacotes Python. Sem USER root o pip3 retorna 'Permission denied: /nonexistent'. Em produção, avaliar implicações de segurança de rodar como root.

docker-compose.yml

```
services:
  spark:
    build: .
    volumes:
      - ./app
    stdin_open: true
    tty: true
```

Chave	O que faz
build: .	Constrói a imagem a partir do Dockerfile na pasta atual
volumes: - ./app	Monta a pasta do projeto dentro do container. Mudanças refletem automaticamente
stdin_open: true	Mantém a entrada padrão aberta — necessário para terminal interativo
tty: true	Aloca um terminal. Junto com stdin_open mantém o container vivo

Parte 3 — Gerenciando dependências

Estratégia para projetos novos

Em projetos novos você raramente sabe de antemão todas as libs que vai precisar. O fluxo recomendado é explorar primeiro e fixar depois.

Fluxo recomendado

- 1. Começa com o requirements.txt vazio ou só com o essencial
- 2. Entra no container e instala libs experimentalmente com pip3 install
- 3. Testa e valida que a lib funciona e que vai usar no projeto
- 4. Quando satisfeito, gera o requirements.txt de dentro do container
- 5. Rebuilda a imagem com as novas dependências fixadas

Gerando o requirements.txt de dentro do container

```
# instala libs experimentalmente
pip3 install pandas requests

# quando satisfeito, gera o requirements
pip3 freeze > requirements.txt
```

Como a pasta do projeto está montada como volume (- ./app), o requirements.txt gerado dentro do container aparece automaticamente na sua máquina. Não precisa copiar nada.

pip freeze vs pip-chill

Comando	O que gera	Quando usar
pip3 freeze	Todas as libs instaladas incluindo dependências de dependências. Arquivo grande.	Reprodutibilidade total
pip-chill	Só o que você instalou diretamente. Arquivo limpo e legível.	Projetos onde você quer clareza

Adicionando libs no dia a dia

O que mudou	Comando necessário
Código Python	Nada — volume reflete automaticamente
requirements.txt	docker compose down && docker compose up -d --build
Dockerfile ou docker-compose.yml	docker compose down && docker compose up -d --build
.env	docker compose down && docker compose up -d

Parte 4 — Dev Containers com VS Code

Por que Dev Containers?

Notebooks .ipynb são o formato natural para projetos de dados — rodar célula por célula, explorar dados interativamente. O Dev Containers conecta o VS Code diretamente ao container, permitindo usar notebooks normalmente com o kernel rodando dentro do container.

Pré-requisito

Instalar a extensão Dev Containers da Microsoft no VS Code (Ctrl+Shift+X).

Arquivo de configuração

Criar a pasta .devcontainer na raiz do projeto com o arquivo devcontainer.json:

```
{
  "name": "teste-spark",
  "dockerComposeFile": "../docker-compose.yml",
  "service": "spark",
  "workspaceFolder": "/app",
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-toolsai.jupyter"
      ]
    }
  }
}
```

Chave	O que faz
name	Nome do Dev Container — aparece no canto inferior esquerdo do VS Code
dockerComposeFile	Aponta pro docker-compose.yml do projeto
service	Qual serviço do Compose usar — deve bater com o nome em services:
workspaceFolder	Diretório de trabalho dentro do container — mesmo que WORKDIR
extensions	Extensões instaladas automaticamente dentro do container

Abrindo o projeto no container

- 1. Ctrl+Shift+P no VS Code
- 2. Pesquisa Dev Containers: Reopen in Container
- 3. VS Code reabre conectado ao container
- 4. Canto inferior esquerdo mostra: Dev Container: teste-spark

Selecionando o kernel no notebook

- 1. Abre o arquivo .ipynb
- 2. Clica em Select Kernel no canto superior direito
- 3. Seleciona o Python dentro do container (Python 3.10.12)
- 4. Se pedir pra instalar ipykernel — instala. Estará instalando dentro do container

Após configurar o Dev Container, o ipykernel deve ser adicionado ao requirements.txt via pip3 freeze para não precisar instalar manualmente na próxima vez.

Parte 5 — Comandos e fluxo do dia a dia

Comandos essenciais

Comando	O que faz
<code>docker pull</code>	Baixa uma imagem do Docker Hub para a máquina local
<code>docker build -t .</code>	Constrói uma imagem a partir do Dockerfile na pasta atual
<code>docker compose up -d</code>	Sobe os containers em background
<code>docker compose up -d --build</code>	Sobe os containers rebuildando a imagem
<code>docker compose down</code>	Para e remove os containers
<code>docker compose exec spark bash</code>	Abre terminal dentro do container
<code>docker compose logs</code>	Mostra logs dos containers
<code>docker images</code>	Lista imagens locais
<code>docker ps</code>	Lista containers em execução

Fluxo completo do zero

- 1. Pesquisa e escolhe a imagem no Docker Hub
- 2. `docker pull` para baixar
- 3. Cria Dockerfile, docker-compose.yml, .dockerignore, requirements.txt
- 4. Cria .devcontainer/devcontainer.json
- 5. `docker compose up -d --build` para buildar e subir
- 6. Dev Containers: Reopen in Container no VS Code
- 7. Desenvolve normalmente com notebooks rodando dentro do container
- 8. Quando adicionar libs: `pip3 install` dentro do container, testa, `pip3 freeze`, rebuilda

Dúvidas respondidas

Por que python3 e pip3 em vez de python e pip?

A imagem apache/spark usa python3 como comando padrão — python sozinho não é encontrado. Isso é uma convenção de algumas imagens Linux. Sempre teste com `python3 --version` antes de assumir.

Posso usar .ipynb em vez de .py dentro do container?

Sim — e é o formato recomendado para projetos de dados. Notebooks .ipynb permitem rodar célula por célula, explorar dados interativamente e documentar o processo. A integração via Dev Containers + extensão Jupyter do VS Code resolve isso perfeitamente.

O que é o aviso 'Unable to load native Hadoop library'?

É um warning normal do Spark em ambiente local. Significa que a biblioteca nativa do Hadoop não está disponível e o Spark vai usar a implementação Java pura — funciona normalmente, só com performance um pouco menor. Em desenvolvimento local não é um problema.

Preciso saber todos os comandos Linux do Dockerfile de cor?

Não. Os comandos RUN usam Linux puro — você aprende pesquisando quando precisa. Busca por 'install X debian dockerfile' ou pergunta para uma IA. Com o tempo os mais comuns ficam na memória naturalmente.